Understanding this, we can ask Elasticsearch to explain why the cluster is unhealthy (or why shards are unassigned). We can query the server using the cluster allocation API for an explanation of why a shard is in its current state. The following query, for example, asks for an explanation about the `chats` index.

Listing 14.2 Asking the cluster for a shard failure explanation

```
GET _cluster/allocation/explain
{
  "index": "chats",
  "shard": 0,
  "primary": false
}
```

This query returns a detailed explanation of the state of this index. We already know the replica won't be created or assigned in a single-node server, right? To verify, let's ask the cluster if that's the case. The following snippet shows a condensed response from the server for the query in listing 14.2:

```
{
  "index" : "chats",
  "shard" : 0,
  "primary" : false,
  "current_state" : "unassigned",
  "allocate_explanation" : "cannot allocate because
  ➥ allocation is not permitted to any of the nodes",
  "node_allocation_decisions" : [{
    ...
    "deciders" : [{
      "decider" : "same_shard",
       "explanation" : "a copy of this shard is already allocated to
       ➥ this node ..]"}
      ]
...
}
```

The `current_state` of the `chats` index mentioned in the returned response says it is `unassigned`. The response explains that the server can't allocate a copy of the shard because of the `same_shard` decider. (Check the value of the `deciders` array in the previous snippet.)

The node in the cluster must fill different roles by default: `master`, `ingest`, `data`, `ml`, `transform`, and others. We can, however, specify the node's roles by setting the `node.roles` property with the appropriate roles in the elasticsearch.yml file.

We can index data into our `chats` index and perform search queries on this single node instance. Because there are no replicas, the risk of losing data and creating performance bottlenecks arises. To mitigate this risk, we can add node(s) to expand the cluster.